Datasets

We collected **spectral metrics** (e.g., power-law α\alpha) from **599 models**, each stored as a per-model `.h5` file.
For each model:

- **Alpha tensor** shape: `(num_layers, num_modules)`.
- **num_layers** ranges from **2** to **80**.
- **Module naming** corresponds to architecture type; typical patterns include:
  - **LLaMA-style**: `mlp.down_proj`, `mlp.gate_proj`, `mlp.up_proj`, `self_attn.k_proj`, `self_attn.o_proj`, `self_attn.q_proj`, `self_attn.v_proj`.
  - **GPT-style**: `attn.c_attn_k`, `attn.c_attn_q`, `attn.c_attn_v`, `attn.c_proj`, `mlp.c_fc`, `mlp.c_proj`.
  - **MPT-style**: `attn.Wqkv_k`, `attn.Wqkv_q`, `attn.Wqkv_v`, `attn.out_proj`, `ffn.up_proj`, `ffn.down_proj`, `ffn.gate_proj`.
  - **All projected to a uniform naming rule**
- Missing modules in a given model are stored as NaN in the tensor.

```
short_name,source_model,full_name,tensor_shape,num_layers,num_modules,module_names
111m,,Corianas--111m,10x6,10,6,"[""self_attn.k_proj"", ""self_attn.q_proj"", ""self_attn.v_proj"", ""self_attn.o_proj
13B-BlueMethod,,CalderaAI--13B-BlueMethod,40x7,40,7,"[""mlp.down_proj"", ""mlp.gate_proj"", ""mlp.up_proj"", ""self_a
13B-HyperMantis,,digitous--13B-HyperMantis,40x7,40,7,"[""mlp.down_proj"", ""mlp.gate_proj"", ""mlp.up_proj"", ""self_
13B-Legerdemain-L2,,CalderaAI--13B-Legerdemain-L2,40x7,40,7,"[""mlp.down_proj"", ""mlp.gate_proj"", ""mlp.up_proj"",
13B-Ouroboros,,CalderaAI--13B-Ouroboros,40x7,40,7,"[""mlp.down_proj"", ""mlp.gate_proj"", ""mlp.up_proj"", ""self_att
256_5epoch,,Corianas--256_5epoch,14x6,14,6,"[""self_attn.k_proj"", ""self_attn.q_proj"", ""self_attn.v_proj"", ""self
30B-Lazarus,,CalderaAI--30B-Lazarus,60x7,60,7,"[""mlp.down_proj"", ""mlp.gate_proj"", ""mlp.up_proj"", ""self_attn.k_
Adventien-GPTJ,,digitous--Adventien-GPTJ,28x6,28,6,"[""self_attn.k_proj"", ""self_attn.o_proj"", ""self_attn.q_proj""
Aira-Instruct-124M,,nicholasKluge--Aira-Instruct-124M,12x6,12,6,"[""self_attn.k_proj"", ""self_attn.q_proj"", ""self_
Aira-Instruct-1B5,,nicholasKluge--Aira-Instruct-1B5,48x6,48,6,"[""self_attn.k_proj"", ""self_attn.q_proj"", ""self_at
Aira-Instruct-355M,,nicholasKluge--Aira-Instruct-355M,24x6,24,6,"[""self_attn.k_proj"", ""self_attn.q_proj"", ""self_
```

Model Similarity & Clustering Pipeline

1. Overall Pipeline

Given two models **A** and **B**, each represented by a set of α-value curves (layer-wise) for a set of modules, we compute a similarity score in the following steps:

1. Module Intersection

Let

$$\mathcal{M}_A = \{\text{modules in A}\}, \quad \mathcal{M}_B = \{\text{modules in B}\}$$

We only compare the **overlapping modules**:

$$\mathcal{M} = \mathcal{M}_A \cap \mathcal{M}_B$$

If $|\mathcal{M}| = 0$, the similarity is undefined (`NaN`).

2. Layerwise Pooling (per module)

For each module $m \in \mathcal{M}$:

- Let $\mathbf{x}^{(m)} \in \mathbb{R}^{L_A}$ be the α-values of model A's module m across layers, and $\mathbf{y}^{(m)} \in \mathbb{R}^{L_B}$ for model B.
- Align layers via a **pooling strategy** so that the two vectors have the same length (L):
  - **Mean pooling** over layer bins
  - **Interpolation** to a fixed number of points
  - **First/last k layers** (optional)

After pooling, we get:

$$\tilde{\mathbf{x}}^{(m)}, \tilde{\mathbf{y}}^{(m)} \in \mathbb{R}^{L}$$

3. Per-module Metric

For each module m, compute a metric d_m (or similarity s_m)
on the **pooled curves** using only the positions where both values are finite (masking NaNs):

Example: **Spearman rank correlation**

$$s_m = \rho_{\text{Spearman}}\left(\tilde{\mathbf{x}}^{(m)}, \tilde{\mathbf{y}}^{(m)}\right)$$

which is in $[-1, 1]$.

4. Aggregation Across Modules

Aggregate per-module similarities into a single score:

$$S(A, B) = \begin{cases} \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} s_m, & \text{mean pooling} \\ \max_{m \in \mathcal{M}} s_m, & \text{max pooling} \\ (\text{other pooling strategies}) \end{cases}$$

5. Kernel Conversion (optional)

If the metric produces a correlation in ([-1, 1]) and you want a kernel in ([0, 1]):

$$K(A, B) = \frac{S(A, B) + 1}{2}$$

If using the **basic Spearman rank** metric, we keep the raw value in $[-1, 1]$.

---

2. Independence Metrics

We evaluated four similarity/distance metrics on the **alpha tensors** (spectral metrics per module per layer), designed to handle varying module counts, missing values, and rank-based comparisons. Each metric compares two models' corresponding module curves **layer-by-layer**, then averages results across modules.

2.1 Spearman Correlation Kernel

Computes Spearman's rank correlation (ρ) between two vectors, which measures the monotonic relationship between values while being robust to outliers and scale differences.

$$rho = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2 - 1)}, \quad d_i = \text{rank}(x_i) - \text{rank}(y_i)$$

Kernelized to $[0, 1]$ for similarity:

$$K_{\text{spearman}} = \frac{\rho + 1}{2}$$

2.2 Kendall's Tau Kernel

$$\tau = \frac{n_c - n_d}{\frac{1}{2}n(n-1)}, \quad K_{\text{kendall}} = \frac{\tau + 1}{2}$$

2.3 Rank-RBF Kernel

$$r_x = \text{rank}(x), \quad r_y = \text{rank}(y), \quad d = \|r_x - r_y\|_2$$

$$K_{\text{rank_rbf}} = \exp\left(-\frac{d^2}{2\sigma^2}\right)$$

2.4 Spearman Rank

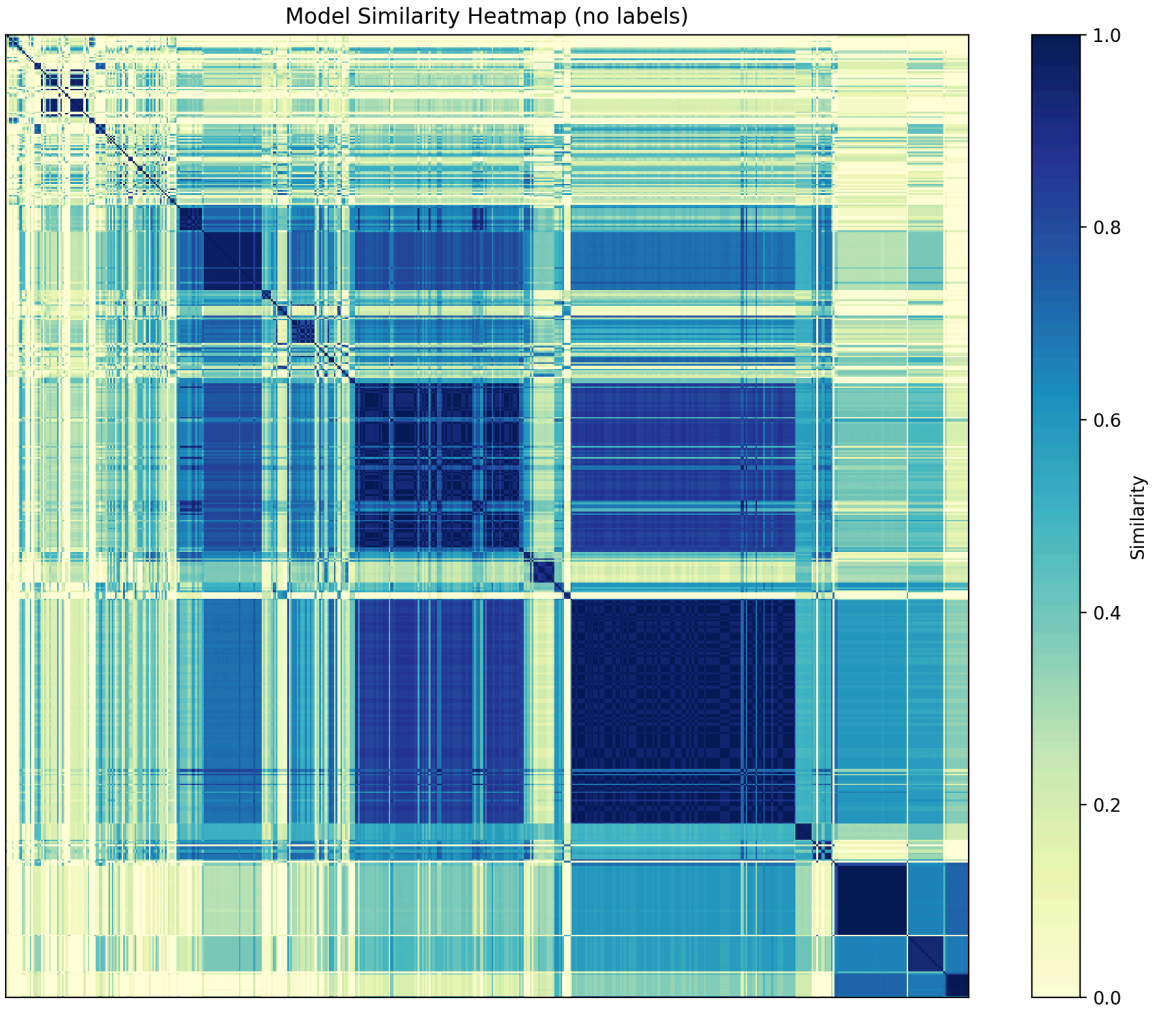
$$\rho = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2 - 1)} \quad (\text{no scaling}; \rho \in [-1, 1])$$

---

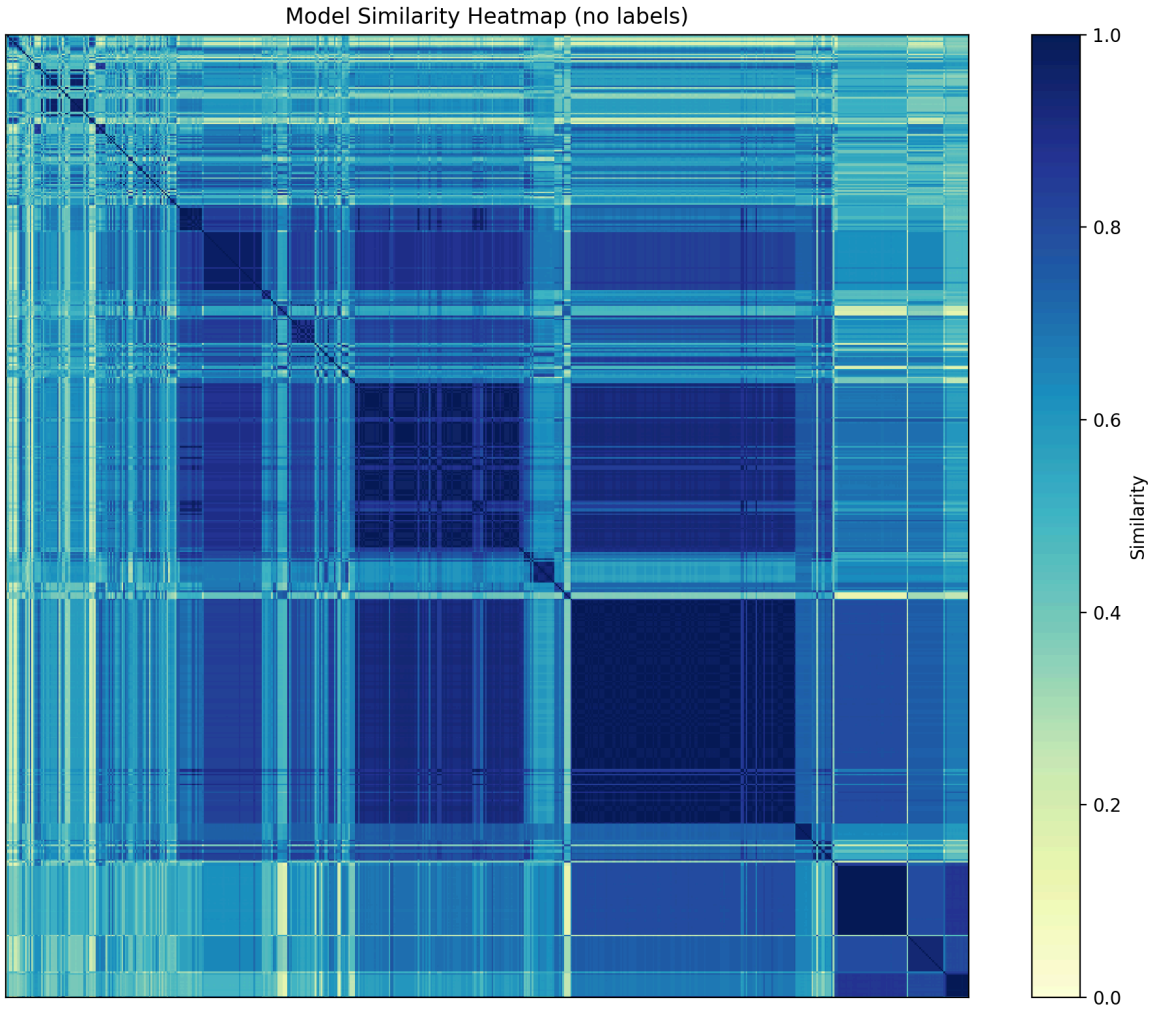
3. Results

We computed full pairwise similarity matrices for all kept models and visualized them as heatmaps under various grouping schemes.

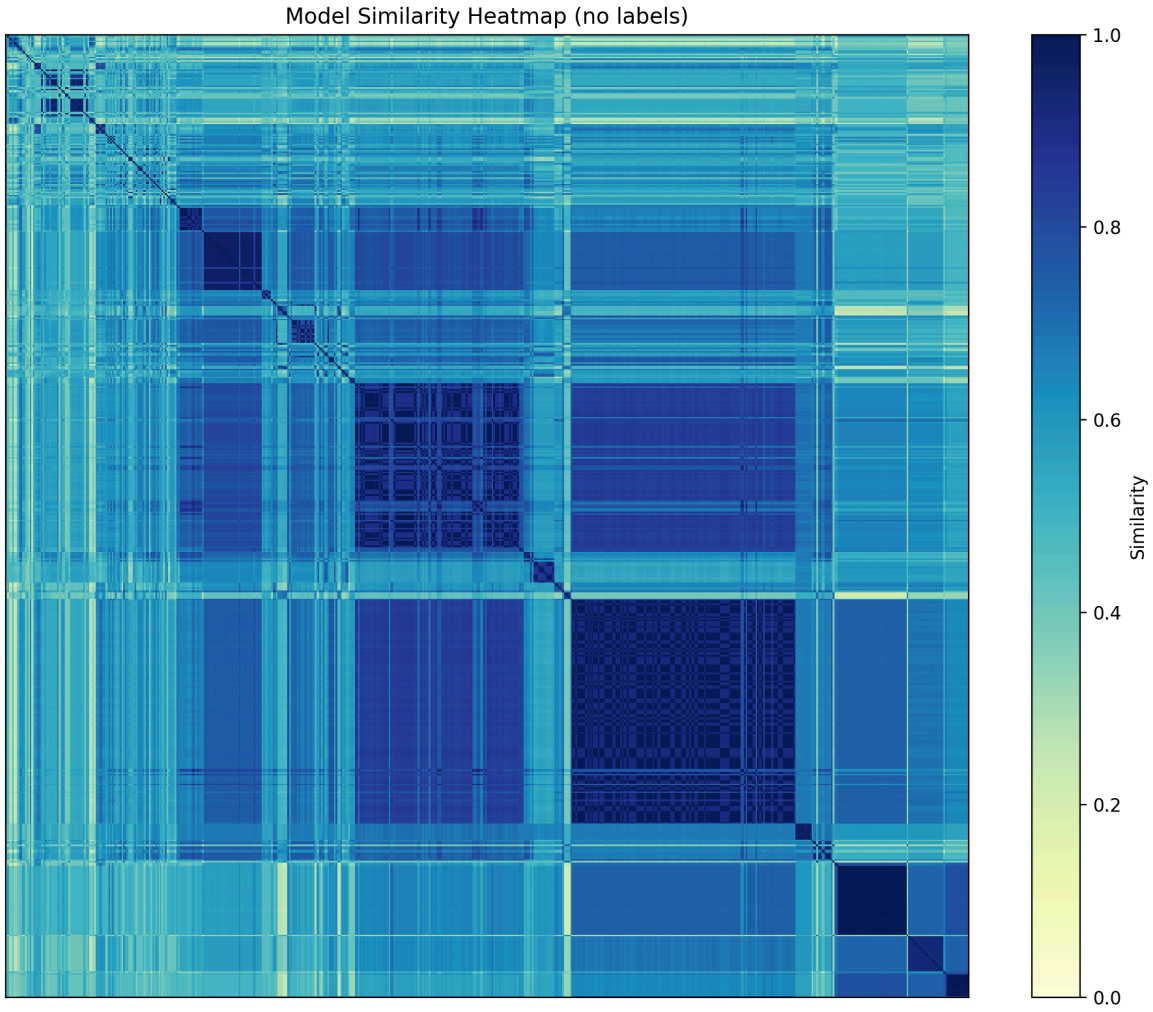
- **Global heatmap** — all models, no labels (due to scale)
  Spearman Rank:



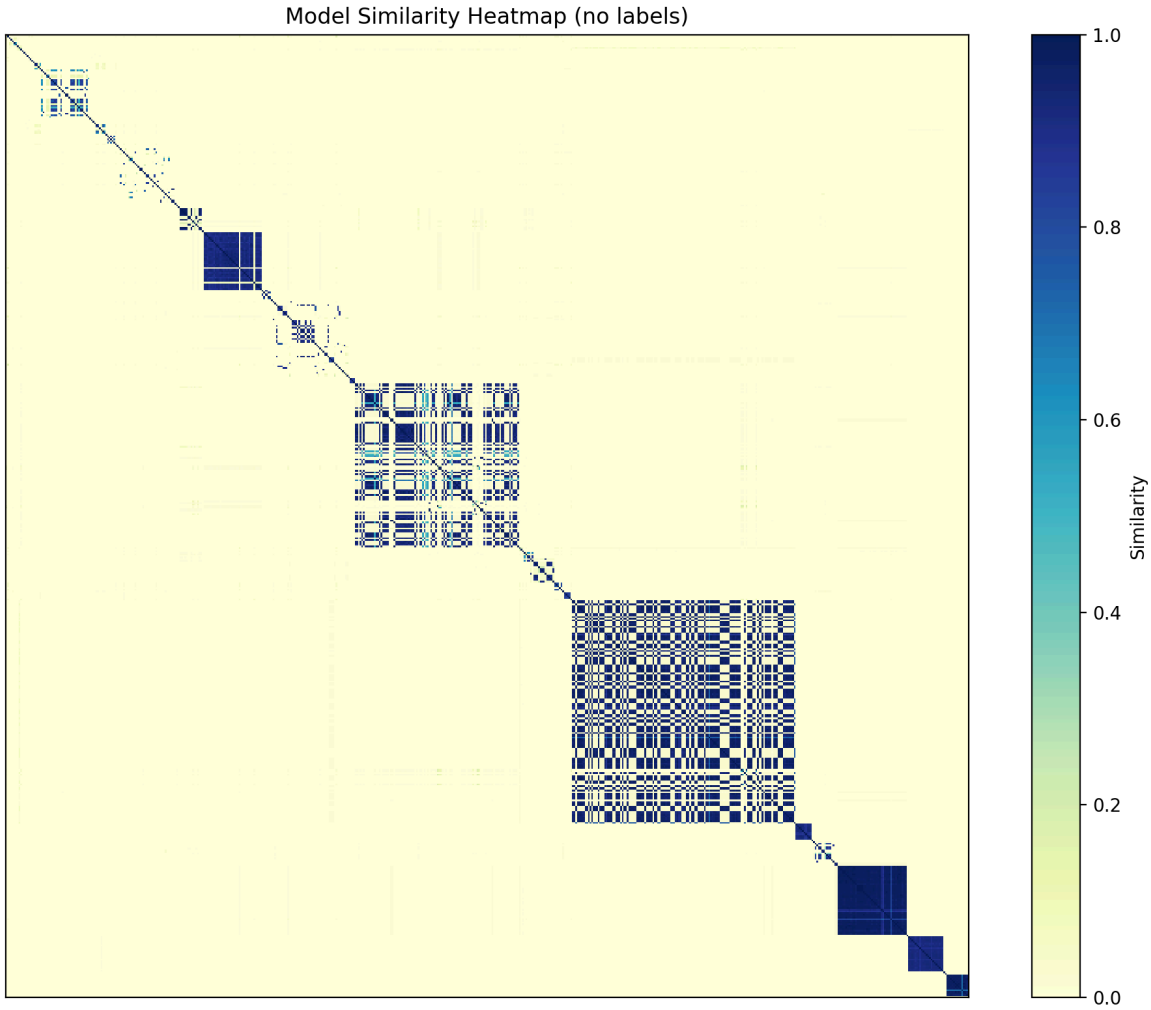
  SR Kernel:


Model Similarity Heatmap (no labels)
1.0
0.8
0.6
0.4
0.2
0.0
Similarity

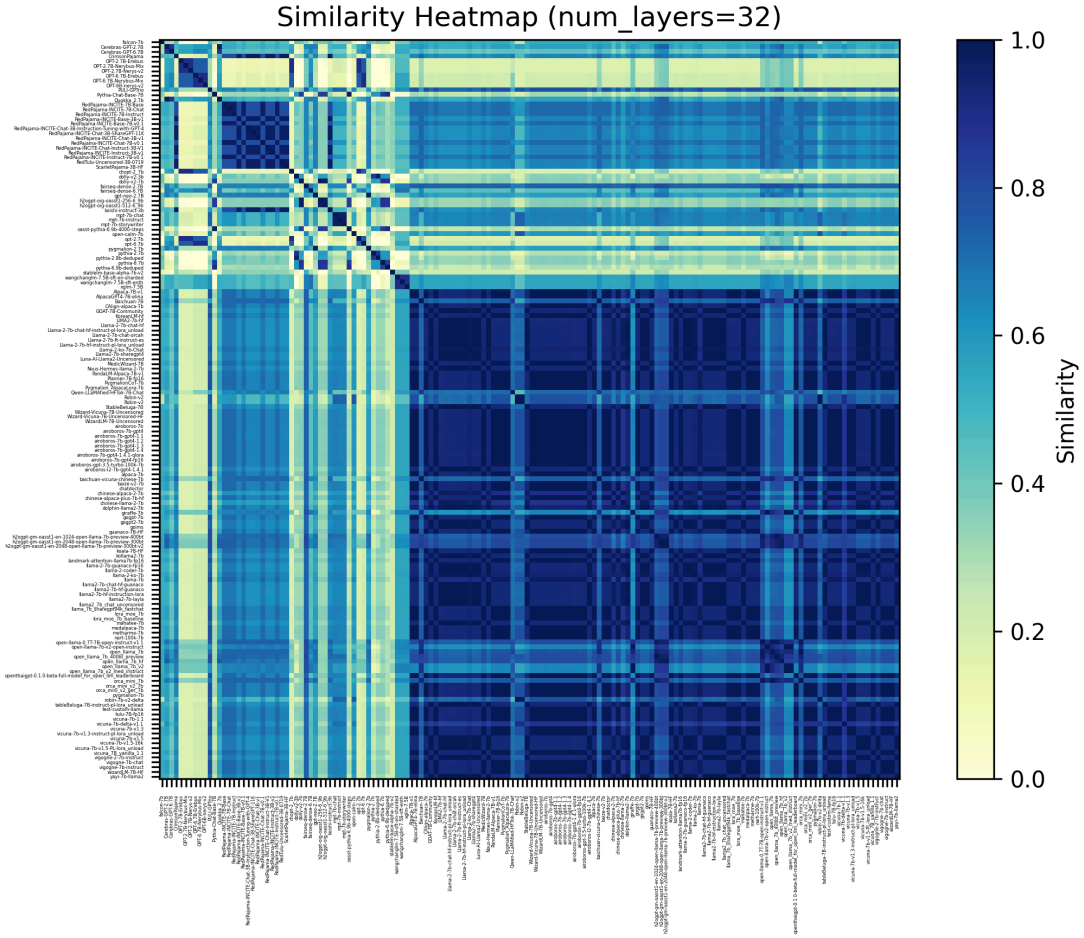
Kendall:


Model Similarity Heatmap (no labels)
1.0
0.8
0.6
0.4
0.2
0.0
Similarity

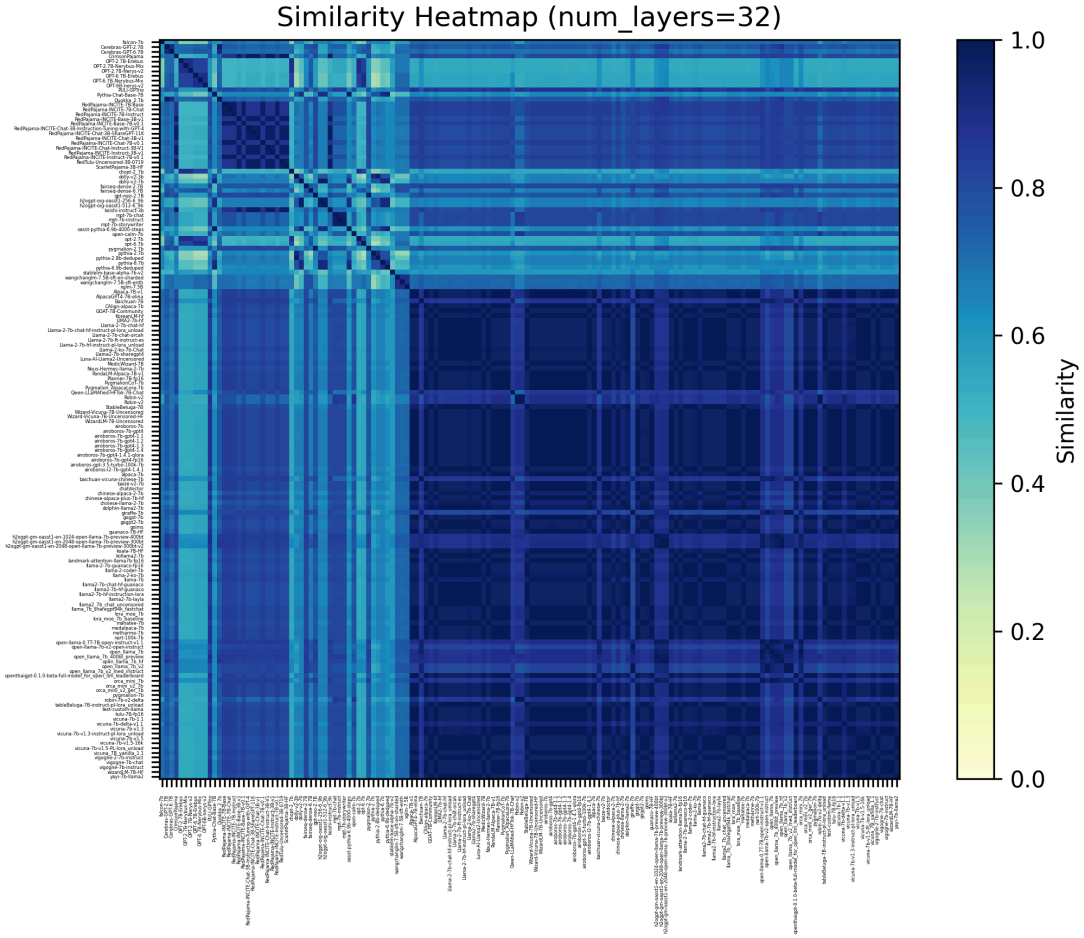
rank RBF:



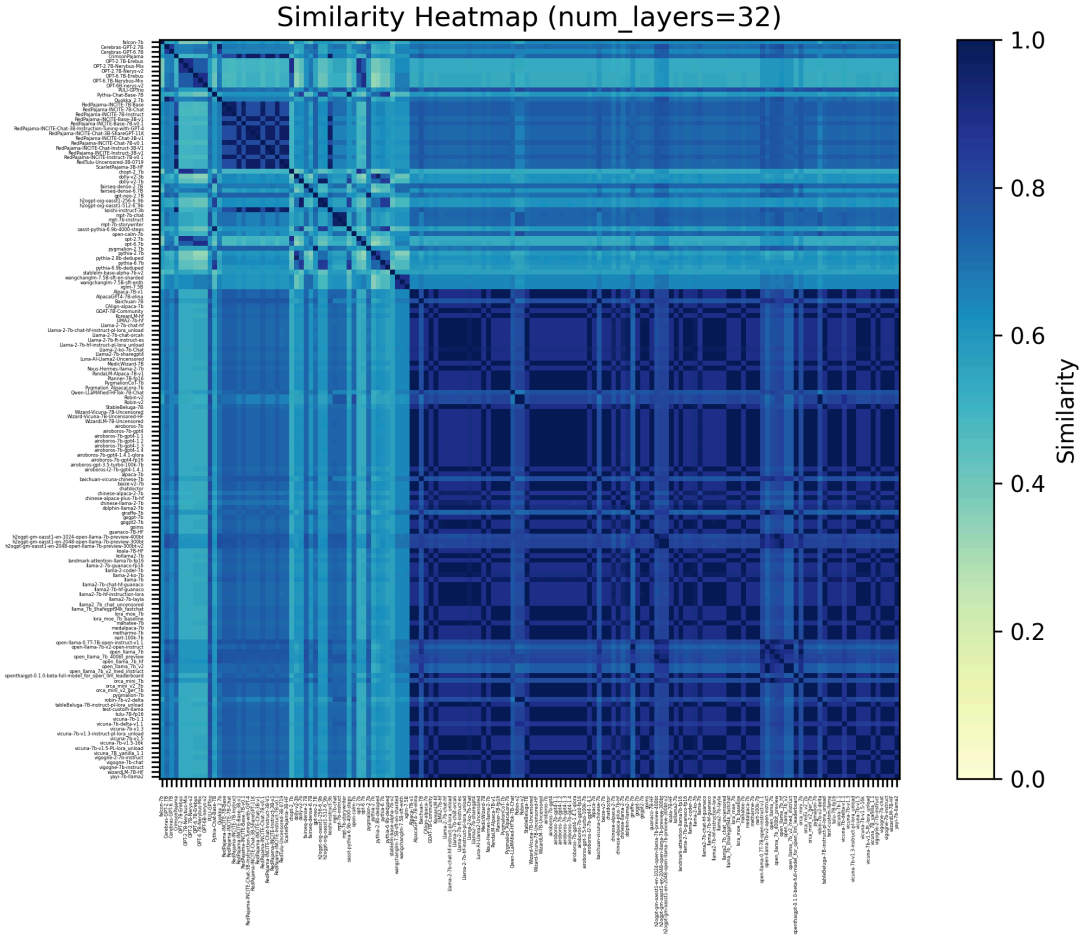
- Models with 32 Layers:
  Spearman Rank:


Similarity Heatmap (num_layers=32)
Similarity
1.0
0.8
0.6
0.4
0.2
0.0

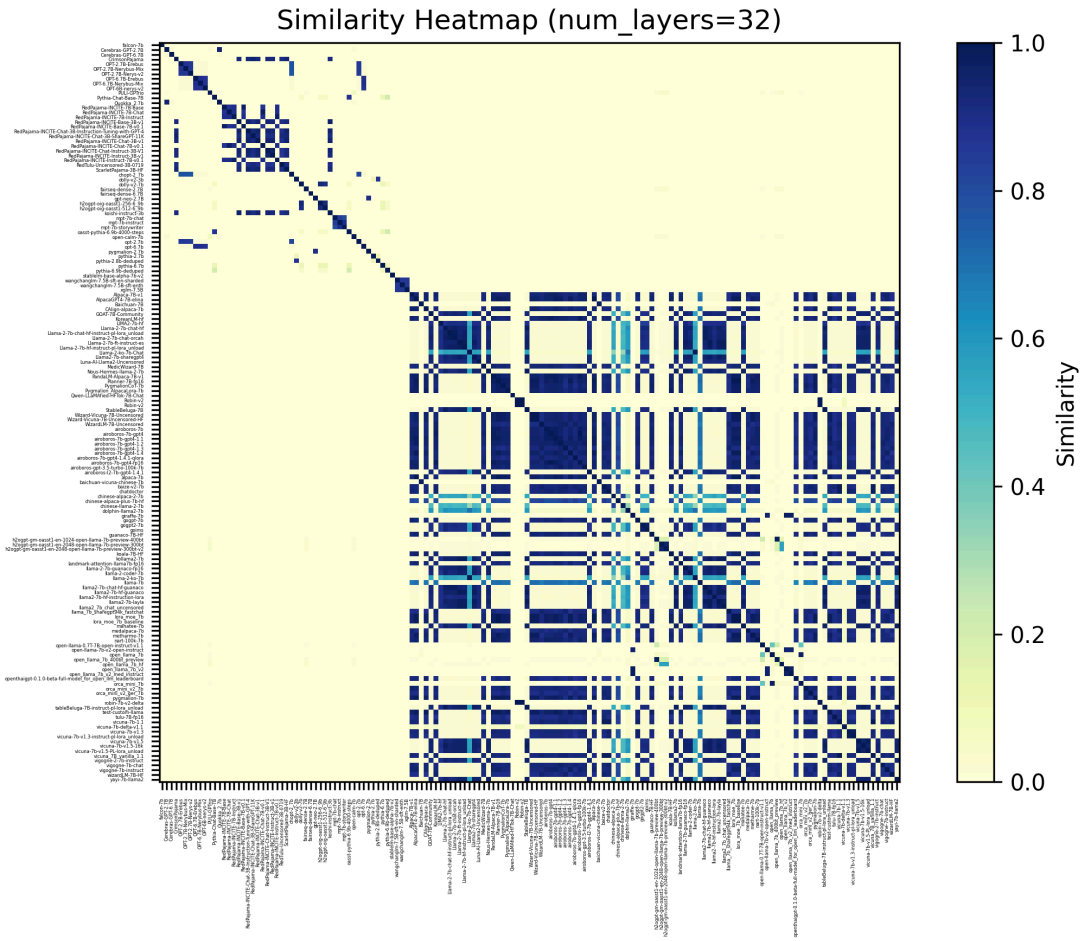
SR Kernel:


Similarity Heatmap (num_layers=32)
Similarity
1.0
0.8
0.6
0.4
0.2
0.0

Kendall:


Similarity Heatmap (num_layers=32)
1.0
0.8
0.6
0.4
0.2
0.0
Similarity

rank RBF:



---

4. Observations & Open Questions

Observations (current run)

1. **No ground-truth labels.** HF cards rarely state the base model, so we cannot use supervised metrics; all conclusions are unsupervised.
2. **Clear clustering emerges.** Many models form dense clusters rather than a sparse, continuous manifold—there *is* structure to exploit.
3. **Rank-RBF looks overly sparse.** The embedding/affinity graph becomes too fragmented; my hypothesis is **over-aggressive bandwidth** (σ too small) or excessive sensitivity to local rank noise → potential overfitting to incidental differences.
4. **Pooling to 32 layers is unverified.** Mapping all models to 32 depth points may introduce alignment artifacts; we haven't shown that K=32 preserves neighborhood relations.
5. **Other two kernels feel under-informative.** Plain Spearman/Kendall seem to suppress useful signal in some modules; clusters are less separated than expected.

**Hypotheses

- **Bandwidth mismatch for Rank-RBF.** σ chosen globally is too small for some modules/datasets → kernel values collapse toward 0 off the diagonal, making the graph sparse.
- **Depth pooling bias.** Uniform resampling to K=32 misaligns patterns whose "functional stages" occur at different relative depths across families; over-smoothing or phase-shifts weaken correlations.
- **Rank-only kernels can be too invariant.** Pure rank correlations may discard magnitude/contrast that's actually discriminative for some modules, reducing separation.
- **Module coverage effects.** Varying module overlap across models skews aggregated similarity—samples with fewer overlapping modules may bias downward (or upward) depending on coverage penalties.

Immediate follow-ups (targeted checks to validate/refute)

1. **Bandwidth sanity for Rank-RBF.**
   - Compare global σ via median-heuristic vs. **local σ** (per-model from k-NN distances).
   - Sweep σ on a log grid (e.g., 0.25×, 0.5×, 1×, 2×, 4× median distance) and track: average off-diagonal kernel value, degree distribution, #connected components.
   - Look for a regime where the affinity graph keeps a single giant component with reasonable average degree (e.g., 10–50).
2. **Pooling robustness.**
   - Re-run with K ∈ {24, 32, 40, 48} and (i) mean-bin pooling vs. (ii) interpolation; compare neighborhood preservation (Trustworthiness/Continuity) and stability (AMI of HDBSCAN labels across K).
   - Try **rank on raw layers** + **DTW/soft-DTW** on ranks (only for a smaller subset) to gauge sensitivity to depth shifts.
3. **Kernel ablations.**
   - Rank vs. z-scored curves: compute Spearman on ranks **and** Pearson on z-scored α to test whether magnitude patterns matter.
   - **Hybrid kernel:** convex combo $K = \lambda K_{\text{rank}} + (1-\lambda) K_{\text{z-score}}$, sweep $\lambda \in [0,1]$, check cluster stability.
4. **Module-level diagnostics.**
   - Per-module separability: distribution of per-module similarities across random pairs; identify modules that are near-constant/low-variance (down-weight) vs. highly discriminative (up-weight).
   - Re-compute aggregated similarity with **idf-style weights** or learned weights from a small validation objective (even unsupervised, e.g., maximize modularity).
5. **Coverage control.**

- Plot similarity vs. overlap ratio $|\mathcal{M}_A \cap \mathcal{M}_B| / |\mathcal{M}_A \cup \mathcal{M}_B|$. If strong correlation exists, tune coverage penalty η^γ (sweep γ).

**Next Steps

We are currently in **Phase 1** (unsupervised similarity mapping).

Phase 2 — Metric Learning with Partial Labels

- Select ~a subset of labeled models to train a supervised metric learner.
- Evaluate whether learned space improves separation of known families.
- Tune thresholds for open-set rejection.

Phase 3 — Open-set Classification & Deployment

- Apply the learned metric to all 599 models.
- Assign family labels to high-confidence matches; mark low-confidence as unknown.
- Validate against new labels as they become available.

Do you want me to also **add a visual table** in Section 1 that directly compares the 4 metrics' pros/cons and output range? That could make it even clearer in a presentation.